
The Islamia University of Bahawalpur

Department of Computer Science



SOFTWARE REQUIREMENTS SPECIFICATION

(SRS DOCUMENT)

for

FixItNow

Version 1.0

By

Muhammad Yar Sanwal

F22BDOCS1M01119

Fall 2022 – 2026

Supervisor

Ms. Iqra Yaqoob

Bachelor of Science in Computer Science

Table of Contents

Version 1.0	1
Revision History	1
Application Evaluation History	2
Introduction	3
Purpose	3
Scope	3
Overall description	3
Product perspective	3
Operating environment	4
Design and implementation constraints	4
Requirement identifying technique	4
Use case diagram	4
Use case description	6
Functional Requirements	12
1. User Management	12
2. Service Booking	13
3. Payment and Ratings	15
4. Communication & Notifications.....	16
Non-Functional Requirements	17
Usability	17
Security	18
Reliability	18
Performance	18
Scalability	18
References	18

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken

Supervised by
Ms. Iqra Yaqoob

Signature _____

Introduction

The introduction provides an overview of the **FixItNow** platform, outlining its purpose, scope, and the problem it aims to solve. It also explains how the **Software Requirements Specification (SRS)** document is organized and how each section contributes to understanding the system. This document clearly defines all **functional and non-functional requirements** required to design, develop, test, and evaluate the FixItNow system. By documenting these requirements in a structured manner, the SRS ensures a common understanding among stakeholders and serves as a reference throughout the system development lifecycle.

Purpose

The purpose of this SRS is to formally document the requirements of *FixItNow*, an on-demand home services platform. The system allows customers to book professional service providers (handymen) for tasks such as electrical repairs, plumbing, cleaning, carpentry, and appliance maintenance. The document serves as a guideline for developers, testers, supervisors, and stakeholders.

Scope

FixItNow will provide the following major capabilities:

1. Customer registration, authentication, and profile management
2. Service provider registration, verification, and skill-based service listing
3. Searching, filtering, and booking services based on category, price, rating, and location
4. Real-time booking status updates (Pending, Accepted, In Progress, Completed)
5. Two-way communication through in-app chat
6. Ratings and feedback system for service providers
7. Admin dashboard for platform monitoring, user verification, and service category management

FixItNow will be available as a **web app (Next.js & NodeJS)** and a **mobile application (React Native)** for both customers and providers.

Overall description

Product perspective

FixItNow is a standalone system built from scratch. It is not an upgrade to an existing system. It follows the architecture of a two-sided marketplace platform consisting of:

1. Customer Module
2. Service Provider Module
3. Admin Module

The system will rely on a cloud-based backend using **Supabase, PostgreSQL**, real-time event handling, storage services, and authentication.

Operating environment

1. **Client Applications:**

Web: Chrome, Firefox, Edge, Safari

Mobile: Android 8+ and iOS 13+

2. **Backend:** Node.js (API routes), Supabase PostgreSQL database

3. **Frameworks:** Next.js for web, React Native for mobile

4. **Hosting Environment:** Cloud-based deployment (e.g., Vercel for web, Supabase cloud)

Example requirement:

OE-1: The system shall operate correctly on all modern web browsers supporting JavaScript and ES6.

.

Design and implementation constraints

1. The system must be implemented using **Next.js, Node.js, Supabase, and React Native** as per project guidelines.
2. PostgreSQL schema must follow normalized design principles.
3. Authentication must use Supabase Auth.
4. Real-time updates must use Supabase Realtime channels.
5. All user media (images, documents) must be stored using Supabase Storage.

Example constraint:

CO-1: FixItNow shall use Supabase PostgreSQL as the primary database engine.

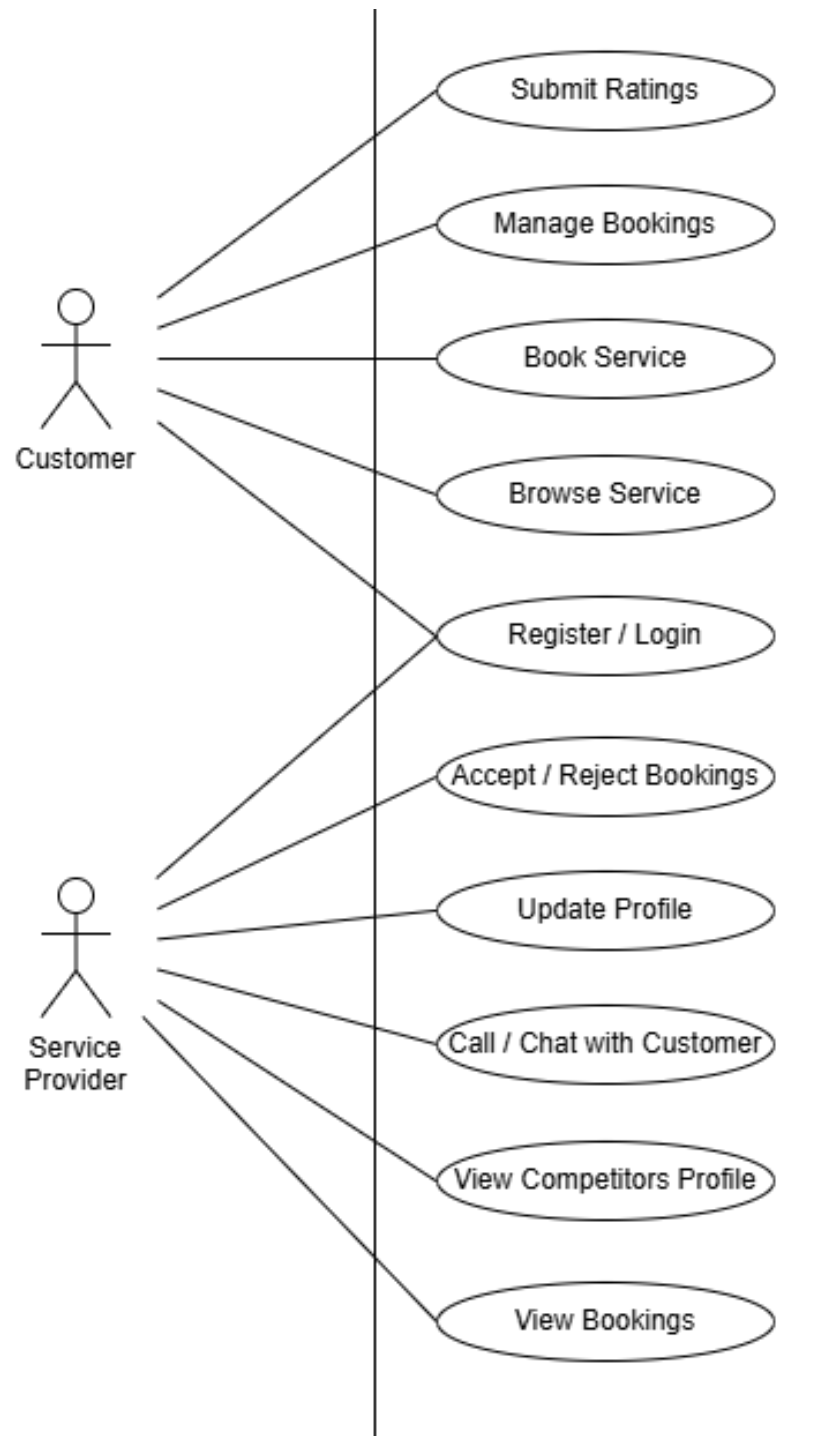
Requirement identifying technique

Use cases are the primary requirement identification technique for FixItNow due to its interactive nature.

Additional techniques used:

1. Actor-goal analysis
2. User stories
3. Event-response analysis for booking workflow
4. Activity flow modeling for complex interactions (booking, acceptance)

Use case diagram



Use case description

The table below indicate a comprehensive use case filled in with the Handy Man Type System.

Use Case ID:	UC-1
Use Case Name:	Register User

Actors:	Customer, Service Provider, Admin
Description:	User creates an account using email and password.

Trigger:	User selects “Register” option
Preconditions:	User must not already be logged in.
Postconditions:	User account is created; profile initialized.
Normal Flow:	1. User opens registration form. 2. User enters personal details. 3. System validates input. 4. System stores data in database. 5. System shows confirmation message. Alternative Flows: • Invalid email → system shows validation error. • Email already exists → system displays message.

Use Case ID:	UC-2
Use Case Name:	Login User

Actors:	Customer, Service Provider, Admin
Description:	User logs into the system.
Trigger:	User enters credentials.
Preconditions:	Registered account must exist.
Postconditions:	User session starts.
Normal Flow:	1. User enters email and password. 2. System validates credentials. 3. System opens user dashboard.

Use Case ID:	UC-3
Use Case Name:	Book Service
Actors:	Customer

Description:	Customer books a service provider based on category and availability.
Trigger:	Customer selects a service.
Preconditions:	Customer must be logged in.
Postconditions:	Booking created, provider notified.
Normal Flow:	1. Customer selects service category. 2. Customer views available providers. 3. Customer selects a provider. 4. Customer enters job description and preferred time. 5. System creates a booking request. 6. Provider is notified. Alternative Flow: ❑ No providers available → system shows “No providers available”.

Use Case ID:	UC-4
Use Case Name:	Accept/Reject Booking
Actors:	Service Provider
Description:	Provider reviews and accepts or rejects the booking.

Trigger:	New booking notification.
Preconditions:	Provider must be logged in..
Postconditions:	Booking status updated.
Normal Flow:	1. Provider opens booking request. 2. Provider presses Accept or Reject. 3. Customer is notified of provider's action.

Use Case ID:	UC-5
Use Case Name:	Chat System
Actors:	Customer, Service Provider
Description:	Users communicate about job details.
Trigger:	Booking reaches "Accepted" status.

Preconditions:	Active booking exists.
Postconditions:	Message stored and delivered.
Normal Flow:	1. User opens chat window. 2. User sends a message. 3. System updates chat in real time.

Use Case ID:	UC-6
Use Case Name:	Rating and Feedback
Actors:	Customer
Description:	After job completion, customer submits a rating.
Trigger:	Booking marked as “Completed”.
Preconditions:	Completed booking exists.

Postconditions:	Rating stored in provider's profile.
Normal Flow:	1. After the Service Provider Provides his service. 2. Customer make the status complete. 3. Customer give the rating and review about his job.

Functional Requirements

Functional requirements specify the behaviors and functionalities the system **must provide**. Each requirement is uniquely identified and includes rationale, source, dependencies, and priority.

1. User Management

Identifier	FR-1
Title	User Registration and Login

Requirement	The system shall allow users (Customer / Service Provider) to register with email or mobile number and password, and log in securely.
Source	Stakeholder meeting & project scope
Rationale	Essential for providing personalized services and tracking bookings.
Dependencies	FR-2 (Profile Management)
Priority	High

Identifier	FR-2
Title	User Profile Management
Requirement	Users shall be able to view and update their profiles including name, contact, address, service skills (providers), and profile picture.
Source	Project specification
Rationale	Enables personalized services and accurate service delivery.
Dependencies	FR-1
Priority	High

2. Service Booking

Identifier	FR-3
Title	Browse and Search Services

Requirement	Customers shall be able to browse or search available service categories (plumbing, cleaning, electrical) and providers based on location, ratings, and availability.
Source	Use case UC2
Rationale	Helps customers choose appropriate services efficiently.
Dependencies	FR-1
Priority	High

Identifier	FR-4
Title	Book Service
Requirement	Customers shall be able to create a service request specifying date, time, service description, and location.
Source	Stakeholder requirements
Rationale	Core functionality of the platform.
Dependencies	FR-3
Priority	High

Identifier	FR-5
Title	Provider Acceptance / Rejection

Requirement	Service Providers shall receive notifications for bookings and can accept or reject them.
Source	Use case UC3
Rationale	Ensures dynamic service assignment and availability control.
Dependencies	FR-4
Priority	High

3. Payment and Ratings

Identifier	FR-6
Title	Payment Integration
Requirement	The system shall allow customers to make payments via online gateways (credit/debit card, mobile wallets) and track payment status.
Source	Stakeholder requirements
Rationale	Ensures smooth financial transactions.
Dependencies	FR-4
Priority	High

Identifier	FR-7
Title	Ratings & Feedback

Requirement	Customers shall be able to rate service providers (1–5 stars) and leave feedback. Providers shall view ratings.
Source	Project specification
Rationale	Encourages quality service and trust.
Dependencies	FR-5
Priority	Meeting

4. Communication & Notifications

Identifier	FR-8
Title	Real-time Chat
Requirement	The system shall allow customers and service providers to chat in realtime for clarifications and service details.
Source	UC4
Rationale	Improves communication and reduces errors.
Dependencies	FR-4
Priority	Medium

Identifier	FR-9
Title	Notifications

Requirement	Users shall receive notifications for booking updates, payment confirmation, service reminders, and new messages.
Source	Project specification
Rationale	Keeps users informed and engaged.
Dependencies	FR-4, FR-8
Priority	High

Non-Functional Requirements

Non-functional requirements (NFRs) define how a system should operate, focusing on its quality attributes and constraints rather than specific functions. Examples include requirements for performance (speed), security, reliability (uptime), usability (ease of use), and maintainability. These requirements are crucial for ensuring a system is efficient, secure, and meets user expectations.

Usability

- The system shall provide a clean, intuitive interface.
- User onboarding shall require no more than 3 steps.
- Navigation must remain consistent across all platforms.

Security

- All passwords shall be stored using secure hashing.
- System shall enforce role-based access control.
- Booking notifications must be delivered within 2 seconds.

Reliability

- System uptime must be at least 99%.
- Failed network operations must retry automatically.

Performance

- 95% of pages must load within 3 seconds under normal network conditions.
- Chat messages must appear within 1 second.
- Booking notifications must be delivered within 2 seconds.

Scalability

- System must support growth in number of customers and providers without performance degradation.

References

- [Supabase Documentation](#)
- [Next.js Documentation](#)
- [IEEE SRS Standard \(IEEE 830\)](#)
- [UML Standards by OMG](#)
- [React Native Documentation](#)